

软件：程序，数据，文档（用户手册等）

软件分类：按层次划分：系统软件，应用软件

按组织划分：开源软件，商业软件 **按结构划分：**单机软件，分布式软件

- 1，所有不满足需求或超出需求的都是缺陷
- 2，没有不存在缺陷的软件，只有迄今为止尚未发现的缺陷

缺陷定义： 非常重要

- + 软件未实现产品说明书要求的功能
- + 软件出现了产品说明书指明不应该出现的功能
- + 软件实现未实现产品说明书虽未提及但应该实现的目标
- + 软件难以理解、不易使用、运行缓慢或者用户体验差

需求必须明确；

软件质量保证部门：SQA

软件测试：

- 1，检查代码和功能实现是否正确
- 2，测试环境和现实环境的不同
- 3，程序以及相关文档的检查。

确认：通过检查和提供客观证据来证实特定目的的功能或应用是否已经实现

验证：通过检查和提供客观证据来证实指定的需求是否满足

如：确认注册界面存在，验证其账号密码限制条件是否满足

软件测试的目的：

- 1，以最少的人力、物力和时间找出软件中潜在的各种错误和缺陷，保证各种错误和缺陷得以修复，避免软件发布后由于潜在的软件错误和缺陷造成的隐患所带来的商业风险。
- 2，利用测试过程中得到的测试结果和测试信息，作为后续项目开发和测试过程改进的重要输入，避免在将来的项目开发和测试中重复同样的错误。

软件测试的定义：

通过人工操作或者软件自动运行的方式，来测试某个程序或系统，进而检验其是否满足规定的需求或是弄清预期结果与实际结果之间的差别。

	测试	调试
主体	测试人员	开发
目标	找 bug	将错误代码修改正确
方法	等价类，边界值	程序和逻辑算法
主要思路	反向思维	正向思维
对象	软件开发过程中的文档、数据以及代码	代码

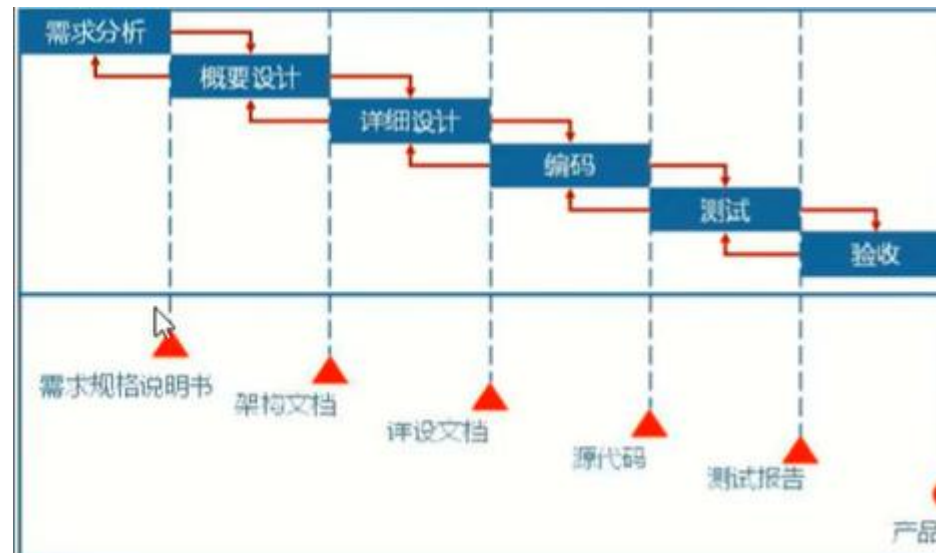
软件工程包括两方面：

软件开发技术：软件开发方法学、软件工具和软件工程环境

软件项目管理：软件质量、项目估算、进度控制、人员组织、配置管理

软件测试是软件质量管理体系中一个非常重要的手段

软件的生命周期

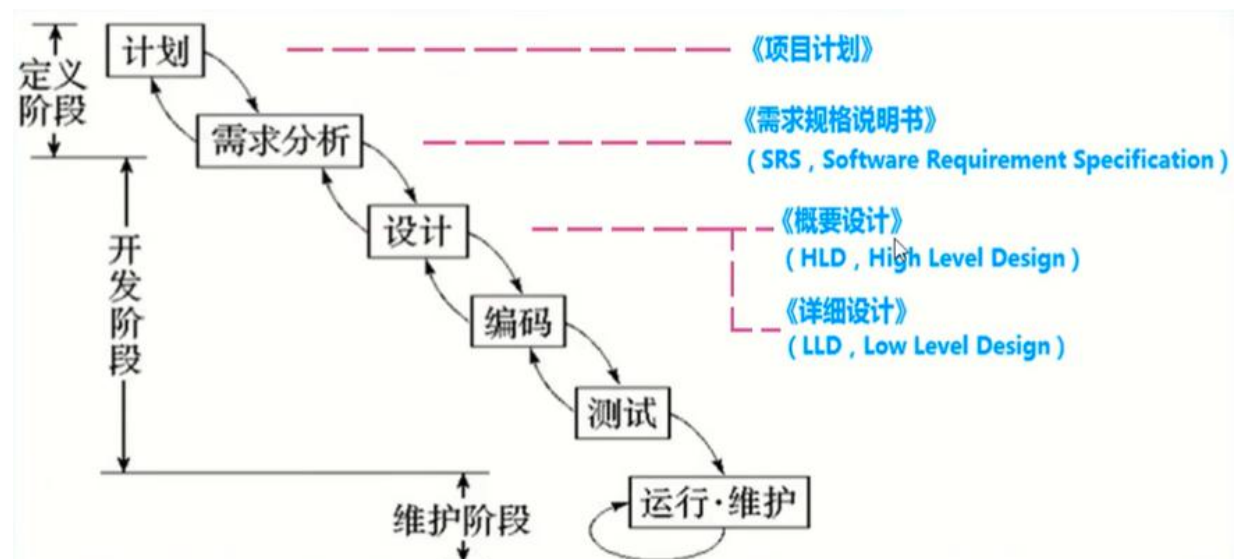


后续更新也需要走这个流程

软件需求规格说明书：SRS

软件开发过程模型：

瀑布模型：最早提出的软件开发模型



存在的问题：1，线性开发，强调时间顺序的严格执行。前阶段不完成，后阶段不开始 2，将测试放在了编码之后。没有体现出测试贯穿软件生命周期的原则，可以避免需求的问题一直延续到代码完成才暴露。

优点：1，为项目提供了按阶段划分的检查点。
2，当前一阶段完成后，只需要去关注后续阶段。

螺旋模型：引入了其他模型不具备的风险分析，适合大型昂贵的系统

迭代模型：强调开发的深入，降低了在一个增量上的开发风险，适应需求的变化

敏捷开发模型：开发需要重点关注，测试了解就行

增量模型：把软件分割成独立的模块，分批次完成和交付。一般和迭代模型共同使用。新增功能：增量；优化性能修复缺陷：迭代模型

快速原型模型：原型：可以模拟操作，简单运行

Axure 软件：制作原型，主要由产品经理完成。然后和用户对齐。

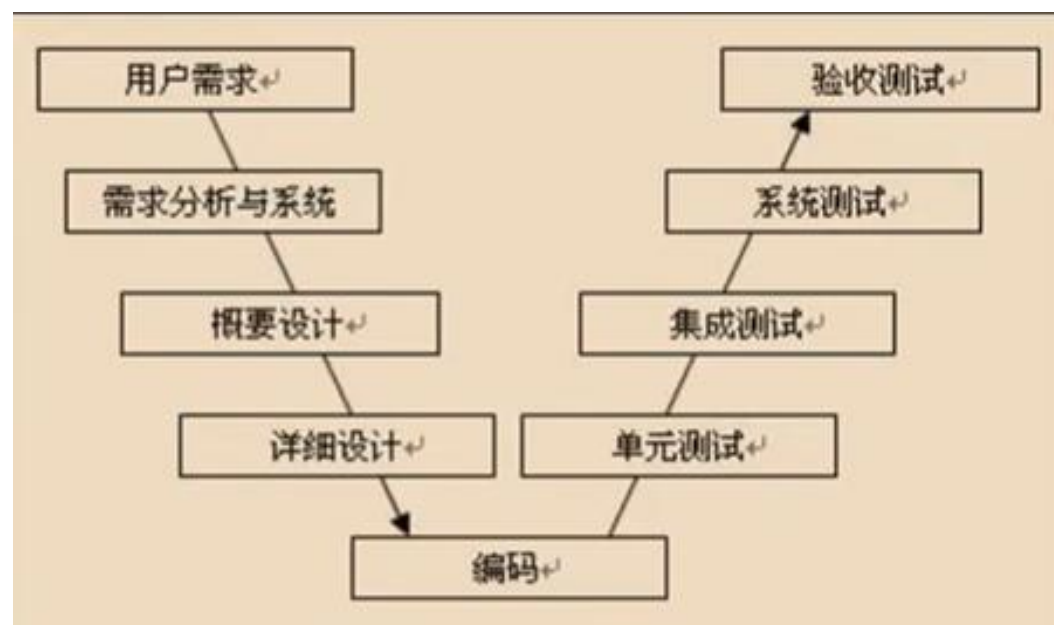
测试过程模型

软件测试流程：获取测试需求——编写测试计划——制定测试方案——开发与设计测试用例——执行测试——提交缺陷报告——测试分析与评审——提交测试总结——准备下一版本测试 必须记住

V 模型：

优点：揭示了开发过程与测试过程中各阶段的对应关系

缺点：没有体现“尽早和不断进行软件测试”的原则；需求的满足情况一直到后期的验收测试才被验证

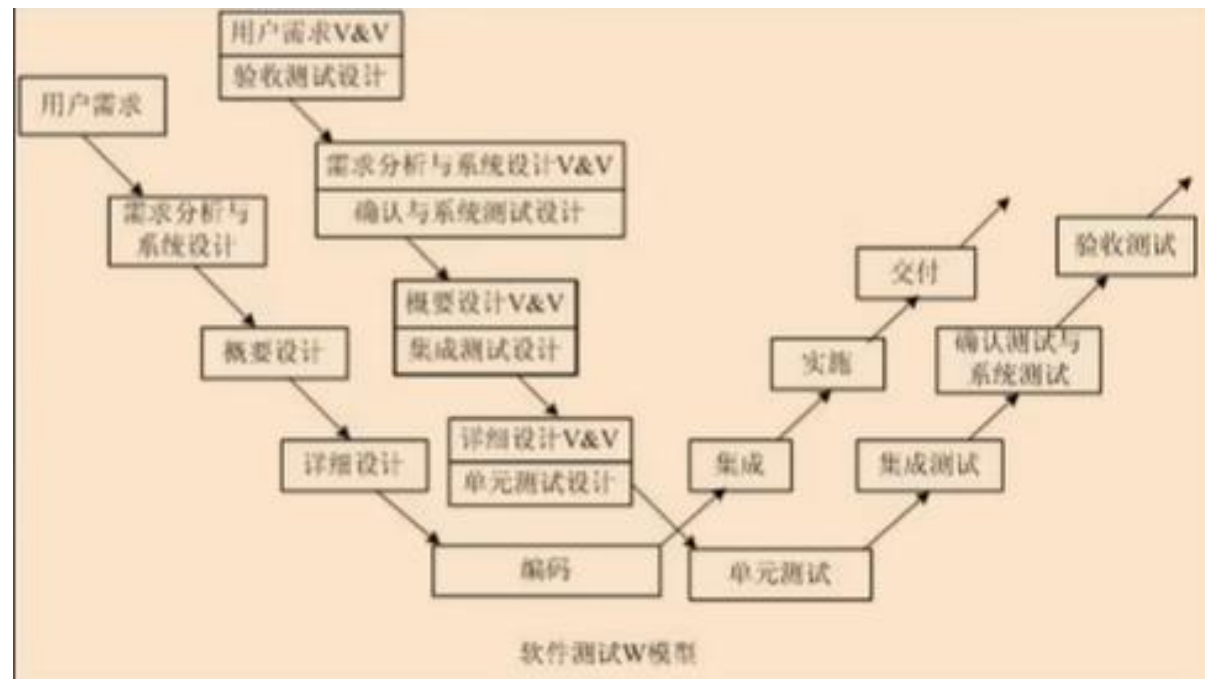


W 模型：两个 V 字型模型，分别代表测试和开发过程

优点：测试活动与软件开发同步进行；测试对象不仅是程序，还包括需求和设计；

能够尽早发现软件缺陷，降低软件开发成本

缺点：需求、设计、编码等活动被视为串行的，无法支持灵活的迭代



H 模型：将测试活动完全独立出来，指出只要某个测试达到准备就绪点，测试执行活动就可以展开

X 模型：定位了探索性测试；即不进行事先计划的特殊类型的测试

软件测试过程理念：尽早测试，全面测试，全过程测试，独立的、迭代的测试

软件测试的分类

1，按照开发阶段划分：

单元测试，一般开发自己完成，对软件设计的最小单元——程序模块进行正确性检验。**集成测试**，比较多涉及到接口测试，学习重点。**确认测试**，验证功能是否实现，也称为冒烟测试。**系统测试**，全面的所有功能的测试。**验收测试**，一般需要有供求双方才存在，三种： α 测试：软件开发商自己进行的交付前的测试； β 测试：软件需求方进行的测试； γ ：第三方的软件测试。

开发不同阶段可以对应不同的测试技术，侧重点也不同

2，按照代码运行划分

静态测试：不实际运行被测对象，静态检查程序代码

动态测试：实际运行被测对象，输入相应的测试数据，检查实际输出结果和预期结果是否相符。

3，按软件特性划分

功能测试：黑盒测试，检查软件的功能是否符合用户的需求，易用性，兼容性。

性能测试：时间空间测试。安全性测试

4，按照测试技术划分

黑盒测试：通过软件的外部表现来发现其缺陷和错误。

白盒测试：通过对程序内部结构的分析、检测来寻找问题。

灰盒测试：介于黑盒和白盒测试之间，关注输出对于输入的正确性，也关注内部表现，但不像白盒测试那样详细、完整（接口测试）。

5，其他测试

回归测试：对软件新版本测试时，重复执行之前某个重要版本的所有测试用例。

冒烟测试：验证软件基本功能是否实现，是否具备可测性。

随机测试：基于经验和直觉的测试，发现一些边缘性的错误。

猴子测试：随便乱点，没有任何主观意识和想法参与进来。

6.，测试主体

手工测试，自动化测试

软件测试的原则

- + 所有测试的标准都是建立在用户需求之上
- + 软件测试必须基于“质量第一”的思想去开展各项工作，当时间和质量冲突时，时间要服从质量
- + 软件项目一启动，软件测试也就开始了，而不是等程序写完，才开始进行测试
- + 穷举测试是不可能的（不能保证软件没有缺陷）
- + 第三方进行测试会更客观，更有效
- + 软件测试计划是做好软件测试工作的前提
- + 测试用例是设计出来的，不是写出来的，所以要根据测试目的，采取相应的方法去设计测试用例，从而提高测试的效率，更多地发现错误，提高程序可靠性。
- + 对发现错误较多的程序段，应进行更深入的测试。一般来说，一段程序中已发现的错误数越多，其中存在的错误概率也就越大。
- + 重视文档，妥善保存一切测试过程文档，如测试计划、测试用例、测试报告
- + 应当把“尽早和不断地测试”作为测试人员的座右铭（软件发布后也要测试）
- + 回归测试的关联性一定要引起充分的注意，修改一个错误而引起更多错误出现的现象并不少见
- + 测试应从“小规模”开始，逐步转向“大规模”
- + 不可将测试用例置之度外，排除随意性
- + 必须彻底检查每一个测试结果
- + 对测试错误结果一定要有一个确认的过程（确认是缺陷才提交）

设计是一项脑力活动；编写是一项体力活动，将设计好的内容通过文字形式表现出来。 缺点不能说：粗心，耐心不够，不善与人沟通，语言表达能力不行 能说：斤斤计较，穷追不舍，说话声音大

测试用例和设计方法

测试用例：设计一个情况，软件程序在这种情况下，必须能够正常运行并且达到程序所设计的预期结果。

测试用例组成：

- 1，测试用例编号
- 2，测试项：一句话表明测试目的
- 3，依赖用例：下游的功能测试依赖于上游的功能测试用例，如登录依赖于注册
- 4，测试步骤：用最朴实语言，写出软件的操作步骤，尽量详细。
- 5，输入数据
- 6，预期结果
- 7，测试结果
- 8，测试人
- 9，备注

测试用例的作用：

- 1，有效性：测试用例是测试人员测试过程中重要的参考依据
- 2，可复用性：测试用例具有重复使用的功能
- 3，易组织性：众多测试用例在未来的测试过程中被创建和使用
- 4，可评估性：测试用例的通过率是检验代码质量的保证
- 5，可管理性：测试用例可作为检验测试进度、工作量及跟踪人员效率的标准

黑盒测试用例设计方法：

测试数据选择：等价类划分法 边界值分析法

测试步骤设计：因果图法 判定表法 正交实验法 功能图法 场景法

等价类划分法：把程序的输入域划分成若干部分，然后从每个部分中选取少数代表性数据作为测试用例。

有效等价类 无效等价类

等价类划分表格分析：以百度账号要求 14 位英文 7 位中文为例

有效等价类	数据	无效等价类	数据
中文、英文混合	肖 haha	数字、特殊字符	12345xhh
14 英文字符	xiaohaha	英文超过 14/中文超过 7	abcdefghijklmn
不能为空	XHH	空	
不能重复	肖浩慧	使用重复的数据进行测试	
7 个中文	慧浩肖		

用例按照测试分类：功能 function、界面 UI、性能 performance、安全 security、接口 interface

冷知识：身份证最后一位是校验码，为 0-9 或 X（罗马的 10）

身份证号倒数第二位是性别，男生为奇数，女生为偶数

边界值分析法:

如果输入条件规定了值的范围,则应取刚达到这个范围的边界的值,以及刚刚超过这个范围边界的值作为测试输入数据。

$6 \leq x \leq 12$, 分别为 5,6,7,11,12,13

$6 < x < 12$, 分别为 6,7,8,10,11,12

三角形分析程序: 输入三个整数值

有效等价类			无效等价类	
不能组成三角形	存在两边和小于第三边	1, 2, 4	小于等于 0	0
	两边和等于第三边	2, 3, 5	小数	3.5
能组成三角形	直角	3, 4, 5	字母	abc
	等腰	4, 4, 5	标点符合	,
	等边	5, 5, 5	汉字	肖哈哈
	锐角	4, 5, 6		
	钝角	2, 3, 4		

因果图法:

因果图法是一种适合于描述对于多种输入条件组合的测试方法;根据输入条件的组合、约束关系和输出条件的因果关系,分析输入条件的各种组合情况,从而设计测试用例的方法。

原因和结果的关系:

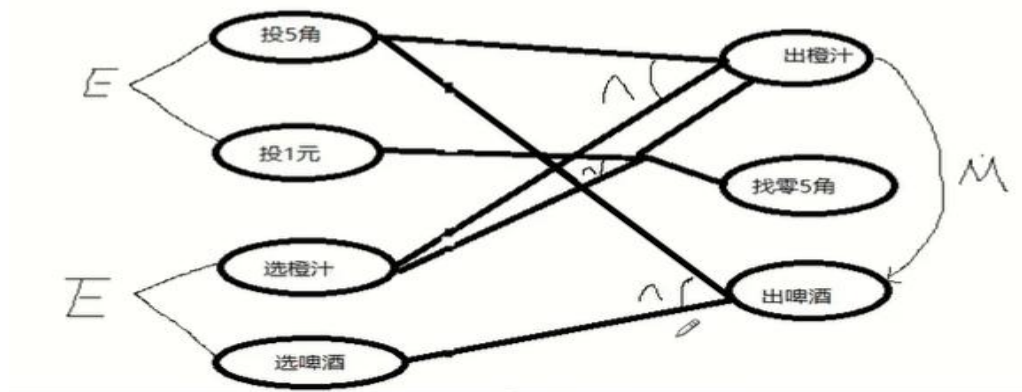
- 1, 恒等。A 成立, B 一定成立
- 2, 非。A 成立, B 一定不成立
- 3, 或。A, B, C 只要一个成立, D 就成立
- 4, 与。A, B, C 都成立, D 才成立。

约束: 1 表示成立。0 表示不成立。

对原因: 互斥 Exclusive: 至多有一个 1。 包含 Include: 至少有一个 1。

唯一 Only: 有且仅有一个 1。 要求 Request: 若 a=1, 则 b 必须为 1

对结果: 屏蔽 Mask: A 结果出现, B 结果一定不出现



因果图法：缺点是原因结果可能逻辑很复杂，关系众多；优点是能够发现设计中存在的不足。

判定表法：多条件的内容组合与结果分析（各条件先后顺序必须互不影响）
针对相同的测试结果，可以合并条件，简化需求

测试用例的设计方法：没有哪一种方式是单独使用的

- 1，所有软件，都是因为某种操作才会导致一定的结果。 ——因果图
- 2，所以软件都有文本框。 ——等价类、边界值法

场景法：

基本流：软件功能按照正确的事件流实现的一条正确的流程。通常一个业务仅存在一个基本流，且基本流仅有一个起点和一个终点。

备选流：除了基本流之外的各支流，包含多种不同的情况。例子：**ATM 机**

正交实验法：由日本统计学家提出，用正交表从大量实验中找到合适的数据组合影响实验结果的量称为：实验因素 每个实验因素的不同取值称为：水平
例子：字的显示效果。因素：字体、字号、颜色 字体选择时：205 个水平

步骤：1，确定因素 2，确定每个因素的水平 3，选择正交表

正交表记为： $L_n(m^k)$ 仅适用于每一种因素的水平数都相同的情况

每一列中，各水平出现的次数相等；

任意多列，组合出现的次数相等；

功能图法：又叫做状态迁徙图；

使用场合：软件状态会根据某些内容、条件、操作变化而变

目标：尽可能覆盖软件的状态、状态条件的组合、状态变迁路径

每种操作在路径上只进行一次

其他测试方法：

测试大纲法，探索性测试法，猴子测试（随意性测试）

用例设计方法综合选择：

缺陷的属性：

缺陷类型 type：功能 function，界面 UI，文档 document，软件包 package，性能 performance，接口 interface

缺陷严重程度 severity：致命 fatal，严重 critical，一般 major，较小 minor

缺陷优先级 priority：取决于缺陷对测试工作的影响程度。立即解决 P1 级，高优先级 P2 级，正常排队 P3 级，低优先级 P4 级

缺陷的严重程度和优先级有什么关系：

- 1，没有任何直接的关系
- 2，不要认为严重的缺陷，修复优先级就高
- 3，如果碰到优先级和严重程度都高的缺陷，也只是偶然

例如：QQ 的帮助按钮，会有经常闪退的现象。严重程度很高，优先级就很低。
企业 logo 错误，不影响任何功能，但必须优先修复。

缺陷状态 status:

激活或打开：由测试人员进行标注；

确认：确认新提交的缺陷是一个真实有效的缺陷

已修复：在缺陷被修复后，一般由开发人员进行

关闭：缺陷被修复完成后，经过测试人员的验证后，没有问题

重新打开：经过测试人员的验证后，缺陷没有修复成功，需要重新打开进行处理

推迟：缺陷现在不修复，推迟到下一个版本或阶段

保留：缺陷暂时修复不了

不能重现：比如闪退，崩溃类型的缺陷，或者由于操作系统的差异、缓存的问题

重复：缺陷已经被其他人提交

需要修改需求说明书：不是功能实现的问题，需求存在问题

缺陷的生命周期：

1 发现缺陷——>2 提交缺陷——>3 确认缺陷——>4 分配缺陷——>5 修复缺陷
——>6 验证缺陷——>7 关闭缺陷

1，2，6，7：测试人员

3，4：测试主管，质量保证 QA，产品经理等

5：相关人员，开发、UI、产品经理等

缺陷的识别：

依据：客观依据：需求文档、设计文档、产品原型、测试用例。

主观依据：同行业类似成熟软件，和开发人员沟通，跟有经验的测试人员沟通，
同行业隐形需求。

测试报告：

缺陷属性：

- 1，缺陷编号。Bug_项目名称_模块名称_功能名称_0001
- 2，所属模块。一级模块/二级模块/三级模块
- 3，优先级。缺陷修复紧急程度。P1>P2>P3>P4
- 4，严重程度。S1>S2>S3>S4
- 5，缺陷概述。一句话描述缺陷的基本情况
- 6，缺陷描述。复现的步骤、预期结果和实际结果列出来
- 7，提交人。
- 8，备注。其他相关信息

目的：展现缺陷的详细信息；展现缺陷的影响程度和方式

缺陷跟踪系统：禅道，ALM，JIRA

获取测试需求是测试工作的重点，也是第一步。通过需求的分析，了解和掌握测试的方向和内容。1) 分析出系统的模块和组织结构。

2) 分析出软件的基本功能和运行流程，包括可能会有哪些人或者哪些角色要用。

3) 识别出软件的重要功能和次要功能。

需求覆盖程度=被测用例覆盖的需求数/需求点总数

最能体现测试人员工作量的指标就是缺陷的数量和用例的数量。

提交的缺陷总量大于等于执行失败的测试用例总量：因为测试时可能出现测试用例之外的缺陷，或者一条测试用例出现多个缺陷。

执行通过的测试用例总量/执行的测试用例数量可以表现出系统的质量是否合格。
执行的测试用例数量/设计的测试用例总量可以表现出系统的需求是否得到满足。

白盒测试：

语句覆盖：

可以使程序中的语句都被测试到，但是它也是覆盖最弱的一种逻辑覆盖方法

分支覆盖，也叫判定覆盖：

运行代码进行测试时，程序中的所有判定语句的真、假分支至少都被执行过一次

条件覆盖：

每个判定语句的所有条件的所有可能结果至少执行一次。

判定条件覆盖：

判定中的每个条件的所有可能取值至少执行一次，并且每个判断本身的所有可能结果至少执行一次

条件覆盖：A=true, A=false, B=true, B=false
C=true, C=false, D=true, D=false,
判断覆盖：A and B =True, A and B =False
C or D =True, C or D =False

案例：

```
if A and B then Action1
if C or D then Action2
```

所以满足条件覆盖和判断覆盖，我们可以设计两条用例：

TestCase1: A=true, B=true, C=true, D=true (条件都为True, A and B=True, C or D =True)

TestCase2: A=false, B=false, C=false, D=false (条件都为False, A and B=False, C or D =False)

组合覆盖，多重条件覆盖：（穷举法）

每个判定中的所有可能的条件结果的组合，以及所有的入口点都至少执行一次

TestCase1: A=True, B=True, C=True, D=True
TestCase2: A=True, B=False, C=True, D=False
TestCase3: A=False, B=True, C=False, D=True
TestCase4: A=False, B=False, C=False, D=False

案例：

```
if A and B then Action1
if C or D then Action2
```

路径覆盖：覆盖程序中所有可能的路径

如果有 n 个判定，则有 2^n 条测试用例。上面就是 16 条

服务器的性能：吞吐量，响应时间，cpu 使用率

软件调试技术包括强行排错法、回溯法、归纳类比法、演绎法、原因排除法。测试的目的是暴露错误，调试的目的是发现并解决错误

单元测试，详细设计文档

集成测试，概括设计文档

系统测试，系统设计文档，需求分析

Beta 测试是验收测试的一种。

α 测试是指软件开发公司组织内部人员模拟各类用户行对即将面市软件产品（称为 α 版本）进行测试，试图发现错误并修正。 α 测试的关键在于尽可能逼真地模拟实际运行环境和用户对软件产品的操作并尽最大努力涵盖所有可能的 用户操作方式。经过 α 测试调整的软件产品称为 β 版本。

β 测试是软件的多个用户在一个或多个用户的实际使用环境下进行的测试。开发者通常不在测试现场，**Beta** 测试不能由程序员或测试员完成。

单元测试依据详细设计，集成测试依据概要设计，系统测试依据需求文档

负载测试：是通过逐步增加系统负载，测试系统性能的变化，并在满足最终确定性性能指标的情况下，系统所能承受的最大负载量的测试

压力测试：逐步增加系统负载，测试系统性能的变化，并最终确定在什么负载下系统性能处于失效状态，并以此来获得系统能提供的最大服务级别的测试

（1）单元测试：

单元测试又称为模块测试，是针对软件设计的最小单位程序模块进行正确性检查的测试工作，单元测试需要从程序内部结构出发设计测试用例，多个模块可以平行地独立进行单元测试。

（2）集成测试

又称为组装测试或联合测试，在单元测试的基础上，需要将所有模块按照概要设计说明书和详细设计说明书的要求进行组装。

（3）确认测试

确认测试的目标是验证软件的功能和性能以及其他特性是否与用户的要求一致。确认测试一般包括有效性测试和软件配置复查。一般有第三方测试机构进行。

（4）系统测试

软件作为计算机系统的一部分，与硬件、网络、外设、支撑软件、数据以及人员结合在一起，在实际或模拟环境下，对计算机系统进行测试，目的在于与系统需求比较，发现问题

（5）验收测试

以用户为主的测试，软件开发人员和质量保证人员参加，由用户设计测试用例。

不是对系统进行全覆盖测试，而是对核心业务流程进行测试。

自顶向下测试：是从程序的初始模块开始测试。

（1）该方***在早期发现顶层的错误。

（2）早期的程序框架可以进行演示

（3）需要开发桩模块辅助测试。有些甚至需要多个桩模块辅助，加大了桩模块本来的错误影响。

（4）测试完一个上层模块后，挑选哪个模块作为下一个测试模块，以及测试的顺序没有唯一的界定标准。

优点：较早地验证了主要控制和判断点；按深度优先可以首先实现和验证一个完整的软件功能；功能较早证实，带来信心；只需一个驱动，减少驱动器开发的费用；支持故障隔离。

缺点：桩的开发量大；底层验证被推迟；底层组件测试不充分。

自底向上测试：是从程序的底层模块开始测试。

（1）I/O 操作可以提前测试，更好提交测试用例。

（2）测试后比较容易观察输出。

（3）需要开发驱动模块。

（4）直到最后一个模块提交，程序才能完整的系统测试。

优点：对底层组件行为较早验证；工作最初可以并行集成，比自顶向下效率高；减少了桩的工作量；支持故障隔离。

缺点：驱动的开发工作量大；对高层的验证被推迟，设计上的错误不能被及时发现。

测试用例是测试程序正确性与否的关键。一个覆盖完全的测试用例可以测试出程序是否正确运行，是否有 bug 等等，是最重要的

测试设计人员：设计测试用例以及设计测试过程、脚本；
制定测试计划由测试经理完成，评估测试活动由测试经理组织召集开发和测试相关的人员来做

动态程序分析是对计算机软件的分析，该计算机软件是通过在真实或虚拟处理器上执行从该软件构建的程序来执行的。

静态分析：

代码覆盖率是由测试需求和测试用例的覆盖或已执行代码的覆盖表示的。建立在对测试结果的评估和对测试过程中确定的变更请求(缺陷)的分析的基础上。

程序数据流分析是一项编译时使用的技术，它能从程序代码中收集程序的语义信息，并通过代数的方法在编译时确定变量的定义和使

TDD，是一种不同于传统 软件开发流程 的新型的开发方法。它要求在编写某个功能的代码之前先编写测试代码，然后只编写使测试通过的功能代码，通过测试来推动整个开发的进行。

集成测试的入口准则：

单元测试已经完成，并提交《单元测试报告》

代码走查完成，已进入受控库并完成产品集成

软件验收测试分为三类：

正式验收测试：

非正式验收测试其中包括 α 测试（由用户、测试人员、开发人员共同参与的内部测试。）和 β 测试（内测后的公测，即完全交给最终用户测试。）

负载测试：在一定的工作负荷下，系统的负荷及响应时间。

强度测试：在一定的负荷条件下，在较长时间跨度内的系统连续运行给系统性能所造成的影响。

容量测试：容量测试目的是通过测试预先分析出反映软件系统应用特征的某项指标的极限值（如最大并发用户数、数据库记录数等），系统在其极限值状态下没有出现任何软件故障或还能保持主要功能正常运行。容量测试还将确定测试对象在给定时间内能够持续处理的最大负载或工作量。容量测试的目的是使系统承受超额的数据容量来发现它是否能够正确处理。容量测试是面向数据的，并且它的目的是显示系统可以处理目标内确定的数据容量。